

UNIT33: GLTF MODEL - SCENE NODES & MESHES

【学習要項】

- ☐ glTF
- ☐ tinyglTF : Header only C++ tiny glTF library(loader/saver)
- ☐ Fetching meshes

【演習手順】

1. tinyglTF ライブラリの導入

※<https://www.khronos.org/registry/glTF/specs/2.0/glTF-2.0.html>

※<https://github.com/javagl/gltf0verview/blob/master/gltf0verview2.0.svg>

① GitHub から tinyglTF ライブラリをダウンロードする

※<https://github.com/syoyo/tinygltf>

② ダウンロードしたファイルをプロジェクトフォルダに展開する

※以下のコードは展開したフォルダが「tinyglTF-release」の場合

2. gltf_model クラスを定義・実装する

① gltf_model クラスの定義（プロジェクトに gltf_model.h を新規追加する）

```
1: #pragma once
2: #define NOMINMAX
3: #include <d3d11.h>
4: #include <wrl.h>
5: #include <directxmath.h>
6: #define TINYGLTF_NO_EXTERNAL_IMAGE
7: #define TINYGLTF_NO_STB_IMAGE
8: #define TINYGLTF_NO_STB_IMAGE_WRITE
9: #include "tinyglTF-release/tiny_gltf.h"
10:
11: class gltf_model
12: {
13:     std::string filename;
14: public:
15:     gltf_model(ID3D11Device* device, const std::string& filename);
16:     virtual ~gltf_model() = default;
17:     struct scene
18:     {
19:         std::string name;
20:         std::vector<int> nodes; // Array of 'root' nodes
21:     };
22:     std::vector<scene> scenes;
23:     int default_scene = 0;
24: };
```

② gltf_model クラスの実装（プロジェクトに gltf_model.cpp を新規追加する）

```
1: #include "gltf_model.h"
2: #define TINYGLTF_IMPLEMENTATION
3: #include "tinyglTF-release/tiny_gltf.h"
4: #include "misc.h"
5:
6: bool null_load_image_data(tinyglTF::Image*, const int, std::string*, std::string*,
7:     int, int, const unsigned char*, int, void*)
8: {
9:     return true;
10: }
11: gltf_model::gltf_model(ID3D11Device* device, const std::string& filename) : filename(filename)
12: {
13:     tinyglTF::TinyGLTF tiny_gltf;
14:     tiny_gltf.SetImageLoader(null_load_image_data, nullptr);
15:
16:     tinyglTF::Model gltf_model;
17:     std::string error, warning;
18:     bool succeeded{ false };
19:     if (filename.find(".glb") != std::string::npos)
20:     {
```

UNIT33: GLTF MODEL - SCENE NODES & MESHES

```
21:     succeeded = tiny_gltf.LoadBinaryFromFile(&gltf_model, &error, &warning, filename.c_str());
22: }
23: else if (filename.find(".gltf") != std::string::npos)
24: {
25:     succeeded = tiny_gltf.LoadASCIIFromFile(&gltf_model, &error, &warning, filename.c_str());
26: }
27:
28: _ASSERT_EXPR_A(warning.empty(), warning.c_str());
29: _ASSERT_EXPR_A(error.empty(), error.c_str());
30: _ASSERT_EXPR_A(succeeded, L"Failed to load glTF file");
31: for (std::vector<tinygltf::Scene>::const_reference gltf_scene : gltf_model.scenes)
32: {
33:     scene& scene{ scenes.emplace_back() };
34:     scene.name = gltf_scene.name;
35:     scene.nodes = gltf_scene.nodes;
36: }
37: default_scene = gltf_model.defaultScene < 0 ? 0 : gltf_model.defaultScene;
38: }
```

3. サンプルモデルの取得

① GitHub から glTF サンプルモデルをダウンロードする

※<https://github.com/KhronosGroup/glTF-Sample-Models>

②ダウンロードしたファイルをプロジェクトフォルダと同じ階層に展開する

4. gltf_model クラスのインスタンス生成

①framework クラスにメンバ変数を定義する

```
std::unique_ptr<gltf_model> gltf_models[8];
```

②framework クラスの initialize メンバ関数で gltf_model オブジェクトを生成する

```
gltf_models[0] = std::make_unique<gltf_model>(device.Get(),
    "..¥¥glTF-Sample-Models-master¥¥2.0¥¥2CylinderEngine¥¥glTF¥¥2CylinderEngine.gltf");
```

③ビルド・実行してアサーションがでないことを確認する

5. ノードデータの取得

① gltf_model クラスに node 構造体とコンテナ変数を定義する

```
1: struct node
2: {
3:     std::string name;
4:     int skin{ -1 }; // index of skin referenced by this node
5:     int mesh{ -1 }; // index of mesh referenced by this node
6:
7:     std::vector<int> children; // An array of indices of child nodes of this node
8:
9:     // Local transforms
10:    DirectX::XMFLLOAT4 rotation{ 0, 0, 0, 1 };
11:    DirectX::XMFLLOAT3 scale{ 1, 1, 1 };
12:    DirectX::XMFLLOAT3 translation{ 0, 0, 0 };
13:
14:    DirectX::XMFLLOAT4X4 global_transform{ 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 };
15: };
16: std::vector<node> nodes;
```

② gltf_model クラスに fetch_nodes メンバ関数を実装する

```
1: void gltf_model::fetch_nodes(const tinygltf::Model& gltf_model)
2: {
3:     for (std::vector<tinygltf::Node>::const_reference gltf_node : gltf_model.nodes)
4:     {
5:         node& node{ nodes.emplace_back() };
6:         node.name = gltf_node.name;
```

```
7:   node.skin = gltf_node.skin;
8:   node.mesh = gltf_node.mesh;
9:   node.children = gltf_node.children;
10:  if (!gltf_node.matrix.empty())
11:  {
12:      DirectX::XMFLOAT4X4 matrix;
13:      for (size_t row = 0; row < 4; row++)
14:      {
15:          for (size_t column = 0; column < 4; column++)
16:          {
17:              matrix(row, column) = static_cast<float>(gltf_node.matrix.at(4 * row + column));
18:          }
19:      }
20:
21:      DirectX::XMVECTOR S, T, R;
22:      bool succeed = DirectX::XMMatrixDecompose(&S, &R, &T, DirectX::XMLoadFloat4x4(&matrix));
23:      _ASSERT_EXPR(succeed, L"Failed to decompose matrix.");
24:
25:      DirectX::XMStoreFloat3(&node.scale, S);
26:      DirectX::XMStoreFloat4(&node.rotation, R);
27:      DirectX::XMStoreFloat3(&node.translation, T);
28:  }
29:  else
30:  {
31:      if (gltf_node.scale.size() > 0)
32:      {
33:          node.scale.x = static_cast<float>(gltf_node.scale.at(0));
34:          node.scale.y = static_cast<float>(gltf_node.scale.at(1));
35:          node.scale.z = static_cast<float>(gltf_node.scale.at(2));
36:      }
37:      if (gltf_node.translation.size() > 0)
38:      {
39:          node.translation.x = static_cast<float>(gltf_node.translation.at(0));
40:          node.translation.y = static_cast<float>(gltf_node.translation.at(1));
41:          node.translation.z = static_cast<float>(gltf_node.translation.at(2));
42:      }
43:      if (gltf_node.rotation.size() > 0)
44:      {
45:          node.rotation.x = static_cast<float>(gltf_node.rotation.at(0));
46:          node.rotation.y = static_cast<float>(gltf_node.rotation.at(1));
47:          node.rotation.z = static_cast<float>(gltf_node.rotation.at(2));
48:          node.rotation.w = static_cast<float>(gltf_node.rotation.at(3));
49:      }
50:  }
51:  }
52:  cumulate_transforms(nodes);
53: }
```

③ gltf_model クラスに cumulate_transforms メンバ関数を実装する

```
1: void gltf_model::cumulate_transforms(std::vector<node>& nodes)
2: {
3:     using namespace DirectX;
4:
5:     std::stack<XMFLOAT4X4> parent_global_transforms;
6:     std::function<void(int)> traverse{ [&](int node_index)->void
7:     {
8:         node& node{nodes.at(node_index)};
9:         XMATRIX S{ XMMatrixScaling(node.scale.x, node.scale.y, node.scale.z) };
10:        XMATRIX R{ XMMatrixRotationQuaternion(
11:            XMVectorSet(node.rotation.x, node.rotation.y, node.rotation.z, node.rotation.w)) };
12:        XMATRIX T{ XMMatrixTranslation(node.translation.x, node.translation.y, node.translation.z) };
13:        XMStoreFloat4x4(&node.global_transform, S * R * T * XMLoadFloat4x4(&parent_global_transforms.top()));
14:        for (int child_index : node.children)
15:        {
```

UNIT33: GLTF MODEL - SCENE NODES & MESHES

```
16:     parent_global_transforms.push(node.global_transform);
17:     traverse(child_index);
18:     parent_global_transforms.pop();
19: }
20: } };
21: for (std::vector<int>::value_type node_index : scenes.at(default_scene).nodes)
22: {
23:     parent_global_transforms.push({ 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1 });
24:     traverse(node_index);
25:     parent_global_transforms.pop();
26: }
27: }
```

④ gltf_model クラスのコンストラクタから fetch_nodes メンバ関数を呼び出す

6. メッシュデータの取得

① gltf_model クラスに buffer_view、mesh 構造体とコンテナ変数を定義する

```
1: struct buffer_view
2: {
3:     DXGI_FORMAT format = DXGI_FORMAT_UNKNOWN;
4:     int buffer{ -1 };
5:     size_t stride_in_bytes{ 0 };
6:     size_t byte_offset{ 0 };
7:     size_t count{ 0 };
8: };
9: struct mesh
10: {
11:     std::string name;
12:     struct primitive
13:     {
14:         int material;
15:         std::map<std::string, buffer_view> vertex_buffer_views;
16:         buffer_view index_buffer_view;
17:
18:         bool has(const char* attribute) const
19:         {
20:             return vertex_buffer_views.find(attribute) != vertex_buffer_views.end()
21:                 && vertex_buffer_views.at(attribute).buffer > -1;
22:         }
23:     };
24:     std::vector<primitive> primitives;
25: };
26: std::vector<mesh> meshes;
27: std::vector<Microsoft::WRL::ComPtr<ID3D11Buffer>> buffers;
```

② gltf_model.cpp に convert_format 関数を実装する

```
1: DXGI_FORMAT convert_format(const tinygltf::Accessor& accessor)
2: {
3:     switch (accessor.type)
4:     {
5:     case TINYGLTF_TYPE_SCALAR:
6:         switch (accessor.componentType)
7:         {
8:         case TINYGLTF_COMPONENT_TYPE_UNSIGNED_BYTE:
9:             return DXGI_FORMAT_R8_UINT;
10:        case TINYGLTF_COMPONENT_TYPE_UNSIGNED_SHORT:
11:            return DXGI_FORMAT_R16_UINT;
12:        case TINYGLTF_COMPONENT_TYPE_UNSIGNED_INT:
13:            return DXGI_FORMAT_R32_UINT;
14:        default:
15:            _ASSERT_EXPR(FALSE, L"This accessor component type is not supported.");
16:            return DXGI_FORMAT_UNKNOWN;
17:        }
18:     }
```

UNIT33: GLTF MODEL - SCENE NODES & MESHES

```
18: case TINYGLTF_TYPE_VEC2:
19:     switch (accessor.componentType)
20:     {
21:     case TINYGLTF_COMPONENT_TYPE_FLOAT:
22:         return DXGI_FORMAT_R32G32_FLOAT;
23:     default:
24:         _ASSERT_EXPR(FALSE, L"This accessor component type is not supported.");
25:         return DXGI_FORMAT_UNKNOWN;
26:     }
27: case TINYGLTF_TYPE_VEC3:
28:     switch (accessor.componentType)
29:     {
30:     case TINYGLTF_COMPONENT_TYPE_FLOAT:
31:         return DXGI_FORMAT_R32G32B32_FLOAT;
32:     default:
33:         _ASSERT_EXPR(FALSE, L"This accessor component type is not supported.");
34:         return DXGI_FORMAT_UNKNOWN;
35:     }
36: case TINYGLTF_TYPE_VEC4:
37:     switch (accessor.componentType)
38:     {
39:     case TINYGLTF_COMPONENT_TYPE_UNSIGNED_BYTE:
40:         return DXGI_FORMAT_R8G8B8A8_UINT;
41:     case TINYGLTF_COMPONENT_TYPE_UNSIGNED_SHORT:
42:         return DXGI_FORMAT_R16G16B16A16_UINT;
43:     case TINYGLTF_COMPONENT_TYPE_UNSIGNED_INT:
44:         return DXGI_FORMAT_R32G32B32A32_UINT;
45:     case TINYGLTF_COMPONENT_TYPE_FLOAT:
46:         return DXGI_FORMAT_R32G32B32A32_FLOAT;
47:     default:
48:         _ASSERT_EXPR(FALSE, L"This accessor component type is not supported.");
49:         return DXGI_FORMAT_UNKNOWN;
50:     }
51:     break;
52: default:
53:     _ASSERT_EXPR(FALSE, L"This accessor type is not supported.");
54:     return DXGI_FORMAT_UNKNOWN;
55: }
56: }
```

③gltf_model クラスに fetch_meshes メンバ関数を実装する

```
1: void gltf_model::fetch_meshes(ID3D11Device* device, const tinygltf::Model& gltf_model)
2: {
3:     HRESULT hr;
4:
5:     size_t gltf_buffer_count = gltf_model.buffers.size();
6:     buffers.resize(gltf_buffer_count);
7:
8:     // Create buffers
9:     for (size_t gltf_buffer_index = 0; gltf_buffer_index < gltf_buffer_count; ++gltf_buffer_index)
10:    {
11:        const tinygltf::Buffer& gltf_buffer = gltf_model.buffers.at(gltf_buffer_index);
12:
13:        D3D11_BUFFER_DESC buffer_desc{};
14:        buffer_desc.ByteWidth = static_cast<UINT>(gltf_buffer.data.size());
15:        buffer_desc.Usage = D3D11_USAGE_DEFAULT;
16:        buffer_desc.BindFlags = D3D11_BIND_INDEX_BUFFER | D3D11_BIND_VERTEX_BUFFER;
17:        D3D11_SUBRESOURCE_DATA subresource_data{};
18:        subresource_data.pSysMem = gltf_buffer.data.data();
19:        hr = device->CreateBuffer(&buffer_desc, &subresource_data, buffers.at(gltf_buffer_index).GetAddressOf());
20:        _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
21:    }
22:
23:    for (std::vector<tinygltf::Mesh>::const_reference gltf_mesh : gltf_model.meshes)
```

```
24:  {
25:      mesh& mesh{ meshes.emplace_back() };
26:      mesh.name = gltf_mesh.name;
27:      for (std::vector<tinyglTF::Primitive>::const_reference gltf_primitive : gltf_mesh.primitives)
28:      {
29:          mesh::primitive& primitive{ mesh.primitives.emplace_back() };
30:          primitive.material = gltf_primitive.material;
31:
32:          // Create index buffer view
33:          if (gltf_primitive.indices > -1)
34:          {
35:              const tinyglTF::Accessor& gltf_accessor{ gltf_model.accessors.at(gltf_primitive.indices) };
36:              const tinyglTF::BufferView& gltf_buffer_view{ gltf_model.bufferViews.at(gltf_accessor.bufferView) };
37:
38:              primitive.index_buffer_view.format = convert_format(gltf_accessor);
39:              primitive.index_buffer_view.buffer = gltf_buffer_view.buffer;
40:              primitive.index_buffer_view.stride_in_bytes = gltf_accessor.ByteStride(gltf_buffer_view);
41:              primitive.index_buffer_view.byte_offset = gltf_buffer_view.byteOffset + gltf_accessor.byteOffset;
42:              primitive.index_buffer_view.count = gltf_accessor.count;
43:          }
44:          // Create vertex buffer views
45:          for (std::map<std::string, int>::const_reference gltf_attribute : gltf_primitive.attributes)
46:          {
47:              const tinyglTF::Accessor& gltf_accessor{ gltf_model.accessors.at(gltf_attribute.second) };
48:              const tinyglTF::BufferView& gltf_buffer_view{ gltf_model.bufferViews.at(gltf_accessor.bufferView) };
49:
50:              buffer_view vertex_buffer_view{};
51:              vertex_buffer_view.format = convert_format(gltf_accessor);
52:              vertex_buffer_view.buffer = gltf_buffer_view.buffer;
53:              vertex_buffer_view.stride_in_bytes = gltf_accessor.ByteStride(gltf_buffer_view);
54:              vertex_buffer_view.byte_offset = gltf_buffer_view.byteOffset + gltf_accessor.byteOffset;
55:              vertex_buffer_view.count = gltf_accessor.count;
56:
57:              primitive.vertex_buffer_views.emplace(std::make_pair(gltf_attribute.first, vertex_buffer_view));
58:          }
59:      }
60:  }
61: }
```

④ gltf_model クラスのコンストラクタから fetch_meshes メンバ関数を呼び出す

⑤ビルド・実行してアサーションがでないことを確認する

※デバッガで gltf_model::meshes の内容を確認しなさい

【評価項目】

- ☐ ノードデータの取得
- ☐ メッシュデータの取得